ATOMIC-LEVEL MATERIALS SIMULATION

Module Code: MES306TC

Kenan Song
School of Chips
Email:
Kenan.Song@xjtlu.edu.cn
Office: C5007

Multi-Orbital Extensions & General TBM Code Architecture

I. The Field Operator Framework

II. Tight-Binding Hamiltonian in Real & Reciprocal Spaces

III. Transition to Code Architecture: The Software Blueprint

I. The Field Operator Framework

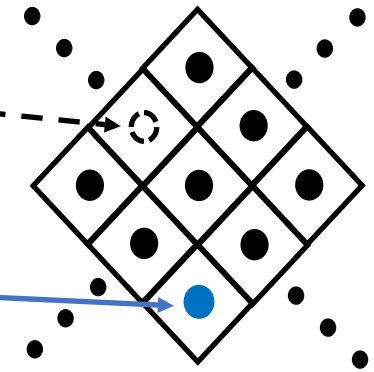
Beyond One-Electron Wavefunctions: Transitioning to Second Quantization

The Old Picture: Wavefunctions $\varphi_\alpha(r - R_n)$ defined in continuous real space. Matrix elements calculated via continuous integration

The New Picture: Define a set of discrete, localized states. Introduce Field Operators that act on the crystal Fock space

Annihilation Operator (c_{jv}): Removes an electron from cell *j*, orbital flavor *v*

Creation Operator ($c_{i\alpha}^{\dagger}$): Creates an electron in cell *i*, orbital flavor *α*



I. The Field Operator Framework

Fermionic Algebras on the Lattice:

Electrons are fermions, which has half integer of the spin

Their lattice field operators must satisfy the Fermionic Anti-Commutation Relations

$$\left\{c_{i\alpha}, c_{j\nu}^{\dagger}\right\} = c_{i\alpha}c_{j\nu}^{\dagger} + c_{j\nu}^{\dagger}c_{i\alpha} = \delta_{ij}\delta_{\alpha\nu}$$

$$\left\{c_{i\alpha}, c_{j\nu}\right\} = 0 \qquad \left\{c_{i\alpha}^{\dagger}, c_{j\nu}^{\dagger}\right\} = 0$$

Physical Meaning: Enforces the Pauli Exclusion Principle perfectly on the discrete tight-binding grid

$$\left(c_{i\alpha}^{\dagger}c_{i\alpha}^{\dagger}\right)|0\rangle = |0\rangle$$

I. The Field Operator Framework

Second Quantized Form: On-Site Potential

Let $\varepsilon_{i\alpha}$ be the single-particle on-site energy of orbital α at lattice site i

The total energy contribution from isolated atoms is written as a diagonal operator

$$H_{on-site} = \sum_{i} \sum_{\alpha} \varepsilon_{i\alpha} c_{i\alpha}^{\dagger} c_{i\alpha} = \sum_{i,\alpha} \varepsilon_{i\alpha} n_{i\alpha}$$

The Number Operator ($n_{i\alpha} = c_{i\alpha}^{\dagger} c_{i\alpha}$): Counts whether an electron is present in orbital α at site i

I. The Field Operator Framework

Second Quantized Form: Kinetic Hopping

An electron 'hopping' from site *j* (orbital *ν*) to site *i* (orbital *α*) is physically described as: Annihilating it at *j*, and Creating it at *i*

$$H_{hopping} = \sum_{\langle i\alpha,j\nu\rangle} t_{i\alpha,j\nu} c^{\dagger}_{i\alpha} c_{j\nu}$$

$t_{i\alpha,j\nu}$ **is the standard real-space hopping integral (amplitudes)**

The true nature of 'hopping' in Quantum Field Theory is that an electron is annihilated at site *j* and simultaneously reborn at site *i*—this is what hopping physically means. Since the physical sequence of operators is annihilation first followed by creation, it is mathematically expressed as $c^{\dagger}_{i} c_{j}$

II. Tight-Binding Hamiltonian in Real & Reciprocal Spaces

The Complete Real-Space Operator Hamiltonian

Combining on-site and directional hopping terms yields the full, generic TBM Hamiltonian operator

$$H = \sum_{i,\alpha} \varepsilon_{i\alpha} c_{i\alpha}^{\dagger} c_{i\alpha} + \sum_{i\alpha \neq j\nu} t_{i\alpha,j\nu} c_{i\alpha}^{\dagger} c_{j\nu}$$

The Hermiticity Constraint: To ensure $H = H^{\dagger}$, the hopping matrix elements must satisfy $t_{i\alpha,j\nu} = t_{j\nu,i\alpha}^{*}$

II. Tight-Binding Hamiltonian in Real & Reciprocal Spaces

Spatial Fourier Transform of Lattice Operators

To exploit lattice translational symmetry, define *k* space creation and annihilation operators via discrete Fourier transform:

$$c_{i\alpha} = \frac{1}{\sqrt{N}} \sum_{k} e^{ik\cdot R_i} c_{k\alpha} \quad ; \qquad c_{i\alpha}^{\dagger} = \frac{1}{\sqrt{N}} \sum_{k} e^{-ik\cdot R_i} c_{k\alpha}^{\dagger}$$

***N* is the number of unit cells; *k* is restricted to the 1st Brillouin Zone**

These new operators retain standard fermionic anti-commutation relations: $\left\{c_{k\alpha}, c_{k'\nu}^{\dagger}\right\} = \delta_{kk'}\delta_{\alpha\nu}$

II. Tight-Binding Hamiltonian in Real & Reciprocal Spaces

Deconstructing the Transformation Proof

Substitute the Fourier expressions of $c_{i\alpha}^{\dagger}$ and $c_{j\nu}$ into the hopping term

$$H_{hopping} = \sum_{i\alpha,j\nu} t_{i\alpha,j\nu} \left(\frac{1}{\sqrt{N}} \sum_{kk'} e^{-ik\cdot R_i} e^{ik'\cdot R_j} c_{k\alpha}^{\dagger} c_{k'\nu} \right)$$

Let $R_j = R_i + b$, and utilize the lattice identity $\frac{1}{\sqrt{N}} \sum_i e^{i(k'-k)\cdot R_i} = \delta_{kk'}$

$$H_{hopping} = \sum_k \sum_{\alpha,\nu} \left[\sum_b t_{\alpha\nu}(b) e^{ik\cdot b} \right] c_{k\alpha}^{\dagger} c_{k\nu}$$

II. Tight-Binding Hamiltonian in Real & Reciprocal Spaces

The *k* Space Matrix Element as a Coefficient Intermediary

Gathering terms, the full Hamiltonian operator becomes beautifully block-diagonalized in *k* space

$$H = \sum_{k} \sum_{\alpha,\nu} H_{\alpha\nu}(k) c^{\dagger}_{k\alpha} c_{k\nu}$$

Those $H(k)$ matrix elements, in the language of second quantization, are precisely the matrix coefficients preceding the k space operator combinations $c^{\dagger}_{k\alpha} c_{k\nu}$

where, the matrix element coefficient $H_{\alpha\nu}(k)$ is defined exactly as

$$H_{\alpha\nu}(k) = \varepsilon_{\alpha}\delta_{\alpha\nu} + \sum_{b} t_{\alpha\nu}(b) e^{ik\cdot b}$$

Diagonalizing the Hamiltonian is, in essence, searching for a new set of quasiparticle operators that forces all these cross-term coefficients to completely vanish

II. Tight-Binding Hamiltonian in Real & Reciprocal Spaces

Enforcing Hermiticity and the '+ *h.c.*' Convention

In literature, to avoid writing long symmetric equations, the Hamiltonian is often truncated

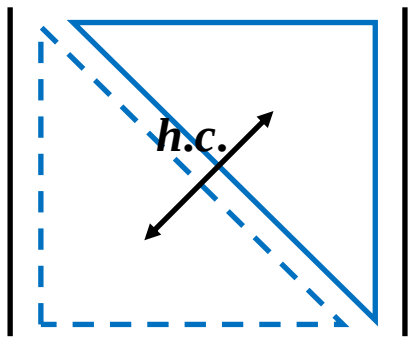
$$H(k) = \sum_{\alpha} \varepsilon_{\alpha} c_{k\alpha}^{\dagger} c_{k\alpha} + \left(\sum_{\alpha<\nu} H_{\alpha\nu}(k)\, c_{k\alpha}^{\dagger} c_{k\nu} + h.c. \right)$$

Hermitian Conjugate

Direct expansion of the *h.c.* part flips the operator order and conjugates the phase

$$\left(H_{\alpha\nu}(k) c_{k\alpha}^{\dagger} c_{k\nu}\right)^{\dagger} = H_{\alpha\nu}^{*}(k) c_{k\nu}^{\dagger} c_{k\alpha} = H_{\nu\alpha}^{*}(k) c_{k\nu}^{\dagger} c_{k\alpha}$$

$H(k) =$



III. Transition to Code Architecture: The Software Blueprint

From Field Operators to Computer Memory Addresses

In code, an operator combination $c^{\dagger}_{i\alpha}c_{j\nu}$ maps directly to a discrete storage location: Hamiltonian(row, cloumn)

Creation Index (i, α) → Matrix Row Index (Row)

Annihilation Index (j, ν) → Matrix Column Index (Column)

A Fortran TBM code is nothing more than an automated counting machine that loops over real space bonds and logs their phase-wrapped hopping strengths into sorted matrix addresses

III. Transition to Code Architecture: The Software Blueprint

Data Structures: Representing Crystal Geometry in Arrays

REAL(KIND=dp) :: latticeinrealspace(3,3) → Stores primitive translation vectors (a_1, a_2, a_3)

REAL(KIND=dp) :: atomcoordinate(3,natom) → Stores internal fractional coordinates (τ_v)

INTEGER :: numorbital(natom) → Allocates the number of active orbitals per atomic species

The very first step in coding is to strictly lock the real-space geometric lattice into standardized arrays within the computer memory

It must be heavily emphasized that one always use the high-precision, dimensionless real type REAL(KIND=dp) across the board to prevent any potential numerical drift

III. Transition to Code Architecture: The Software Blueprint

Neighbor Search Algorithm: The Automated Neighbor Sweep in Real Space

The computer does not know who is bonded to whom so one must execute a spatial distance sweep

Triple Nested Cell Loop: Loop over target replica cells (n_1, n_2, $n_3 \in \{-1, 0, 1\}$)

Distance Formula: $R = \left|R_{replica}(n_1, n_2, n_3) + \tau_j - \tau_i\right|$

Filtering: If $R_{min} - \delta < R < R_{min} + \delta$**, record this pair into the Neighbor List**

This 'neighbor-searching' process only needs to be executed once during the initialization stage of the entire program run. Once the nearest neighbors are successfully swept, a comprehensive Neighbor List is obtained

III. Transition to Code Architecture: The Software Blueprint

Data Anatomy of the Recorded Neighbor List

To avoid re-calculating geometry inside the *k* loop, the Neighbor List stores pre-computed values for each bond

POINTER :: atompointer_i, atompointer_j → Integer pointers to the interacting atoms

REAL(KIND=dp) :: atomdistance(3) → The real-space vector R_j - R_i

REAL(KIND=dp) :: dirctioncosines(3) → Pre-calculated (*l*, *m*, *n*) for instant Slater-Koster lookup

Since the reciprocal-space loop might scan thousands of *k* points, recalculating distances and performing square roots (to find direction cosines) inside the *k* loop would severely degrade the program's performance

Pre-computing and caching these values during initialization is a classic 'space-for-time' code optimization strategy

III. Transition to Code Architecture: The Software Blueprint

Reciprocal Space Scanning: The *k* Path Generator

Energy bands are evaluated along distinct lines connecting high-symmetry vertices in the 1st BZ

Linear Interpolation Algorithm $k(t) = k_{start} + \frac{t}{total\ number\ of\ k}(k_{end} - k_{start})$

The output is a flattened 2D array kpoint(3, Total Number of *k*) representing the ordered path

III. Transition to Code Architecture: The Software Blueprint

Core Loop: Assembling $H(k)$ via Phase Accumulation

For each individual vector kpoint in the path, one initialize a fresh matrix hamiltonian = CMPLX(0.0_dp, 0.0_dp)

The Accumulation Loop

```
DO k = 1, total_bonds
    phase = dot_product(kpoint, neighbor_list(k)%rvector)
    phase_factor = cmplx(cos(phase), sin(phase), kind=dp)
    hamiltonian(i,j) = hamiltonian(i,j) + hopping * phase_factor
END DO
```

Prior to diagonalization, one must recalculate and reassemble $H(k)$ from scratch for every single k point. This is because the phase factor $e^{ik\cdot R}$ heavily depends on k

Utilizing Euler's formula, cos(phase) + (0.0_dp, 1.0_dp) * sin(phase) or using the explicit complex constructor, is the most robust way to build complex numbers in Fortran

III. Transition to Code Architecture: The Software Blueprint

Implementing the '+ *h.c.*' in Numerical Code

In the assembly loop, one only needs to explicitly calculate bonds where row_index <= col_index (the upper triangle)

Calculating the phase exponential is computationally expensive (cos and sin calls)

Leave the lower triangle empty during assembly, and let the subsequent numerical solver know that the matrix is inherently Hermitian. This cuts our assembly time exactly in half

Writing it as '+ *h.c.*' in second quantization directly equates to looping over and assembling only the upper triangle of the matrix in Fortran programming

This is a highly advanced and practical performance optimization technique in scientific computing

III. Transition to Code Architecture: The Software Blueprint

Numerical Diagonalization via LAPACK library

For the complex Hermitian matrix $H(k)$, invoke the standard ZHEEV routine in LAPACK library

Key Arguments

'V': Compute both sorted eigenvalues and eigenvectors

'U': Read only from the Upper triangle of the assembled matrix (enforcing Hermiticity and automatically accounting for the *h.c.* part)

Invoking LAPACK's ZHEEV (Complex Hermitian Eigenvalues) routine is the gold standard across the computational materials science community

Pay close attention to the 'U' argument—it flawlessly instructs the solver to read the upper triangle and automatically handle the lower triangle via mirror conjugate symmetry

III. Transition to Code Architecture: The Software Blueprint

Data I/O: Streaming out Eigenvalues for Plotting

Inside the *k* loop, after ZHEEV finishes, eigenvalues are returned in a sorted real array eigenvalue(D)

File I/O Operation: Write the scalar path index and all D eigenvalues into a structured text file

```
WRITE (UNIT=file_number, FMT='(F10.5, 8F12.6)') kdistance, eigenvalue(1:D)
```

This data file can be instantly rendered using Gnuplot, Python's Matplotlib or Matlab

III. Transition to Code Architecture: The Software Blueprint

Complete Architectural Flowchart

The Global Structural Pipeline

Flowchart

[Geometry Data] → [One-Time Neighbor List Sieve (Real Space)]

[Generate k Path] → [Loop k: Assemble $H_{upper}(k)$ → ZHEEV('U') → Write to Disk]



That's all for this session

Thank you for your attention and great participation